Dimensionality Reduction and Classification: Iris and Indian Pines

Prepared for: Dr. Vineetha Menon
Prepared by: Azaria A. Reed
Date: March 29, 2026

Data Visualization

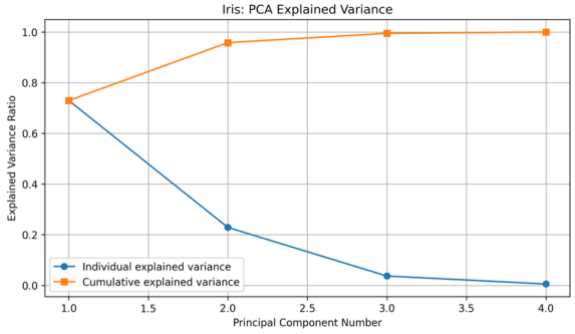

Figure 1. PCA explained variance for the Iris dataset.

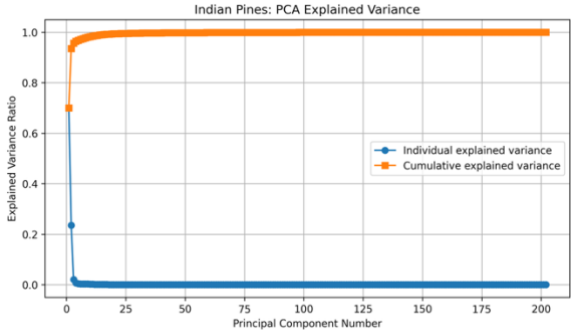

Figure 2. PCA explained variance for the Indian Pines dataset.

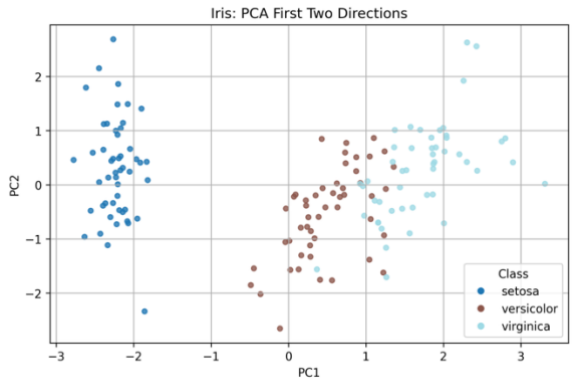

Figure 3. PCA two-dimensional projection for the Iris dataset.

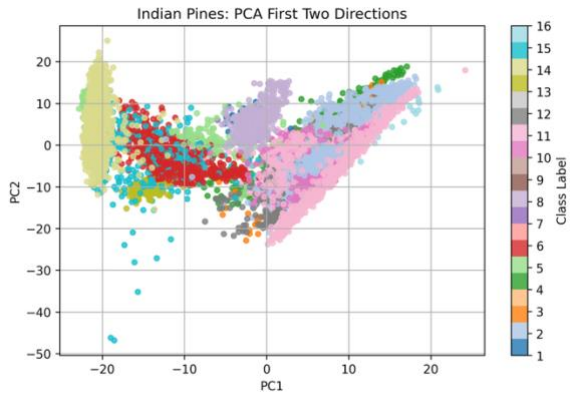

Figure 4. PCA two-dimensional projection for the Indian Pines dataset.

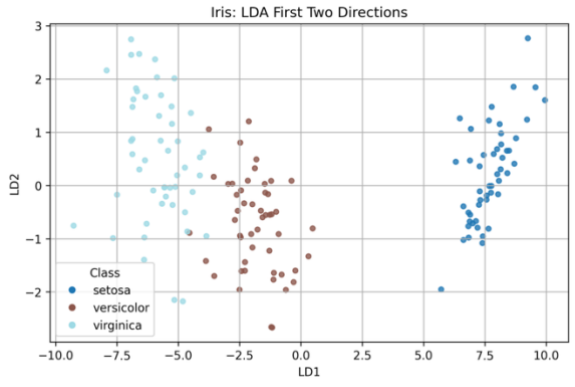

Figure 5. LDA two-dimensional projection for the Iris dataset.

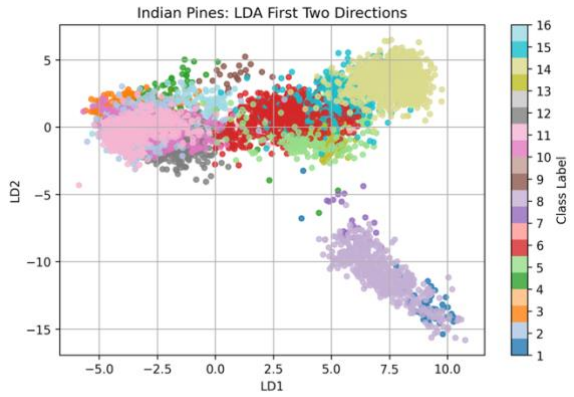

Figure 6. LDA two-dimensional projection for the Indian Pines dataset.

Analysis of Visualizations

Iris Dataset

The Iris dataset shows strong class separability after dimensionality reduction. Figure 1 indicates that the first two PCA components capture nearly all variance, supporting k=2, with further components adding little value. The PCA projection in Figure 3 shows partial separation, while the LDA projection in Figure 5

reveals clear class boundaries. LDA uses label information to maximize separability, producing tighter clusters and less overlap. Therefore, LDA outperforms PCA on the Iris dataset by aligning projections directly with class distinctions rather than merely with variance.

Indian Pines Dataset

In contrast, the Indian Pines dataset is more complex and high-dimensional, yielding different behavior. Figure 2 shows that PCA requires many components to capture the total variance, though the key structure appears early. Selecting k=2 ensures consistency across datasets and captures primary variance directions. The PCA projection in Figure 4 reveals significant class overlap, showing poor separability. LDA in Figure 6 improves clustering by using class labels, but overlap remains due to class similarity and noise in hyperspectral data. While dimensionality reduction is necessary to simplify this dataset, LDA only partly addresses separability. LDA consistently outperforms PCA by directly optimizing class separation, but neither achieves complete separation.

Data Visualization

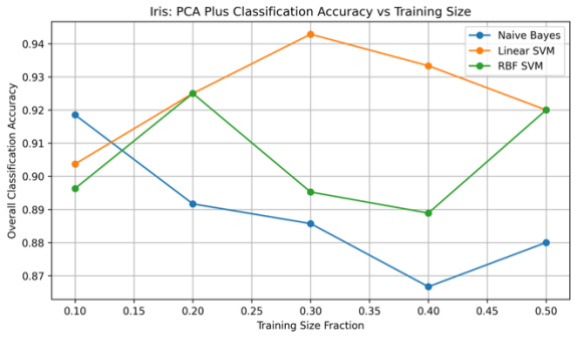

Figure 7. Classification accuracy versus training size for the Iris dataset using PCA plus classification.

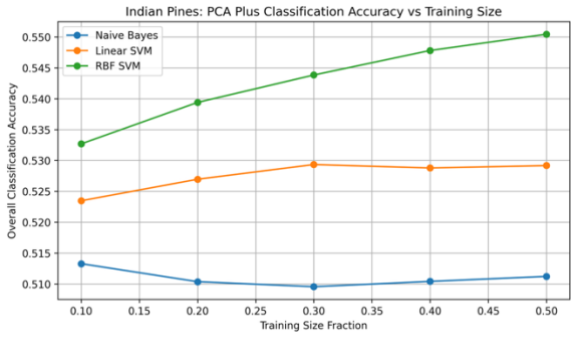

Figure 8. Classification accuracy versus training size for the Indian Pines dataset using PCA plus classification.

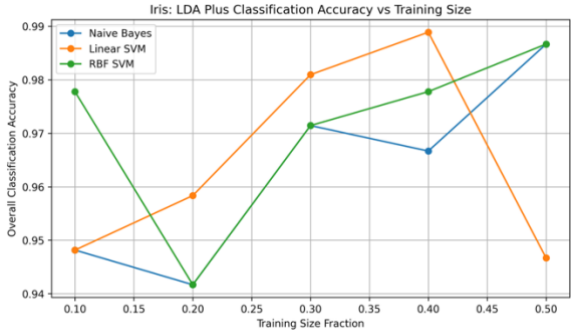

Figure 9. Classification accuracy versus training size for the Iris dataset using LDA plus classification.

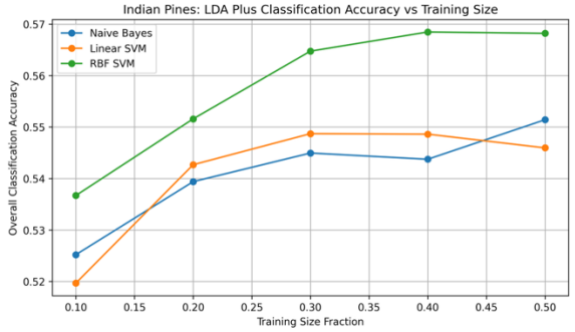

Figure 10. Classification accuracy versus training size for the Indian Pines dataset using LDA plus classification.

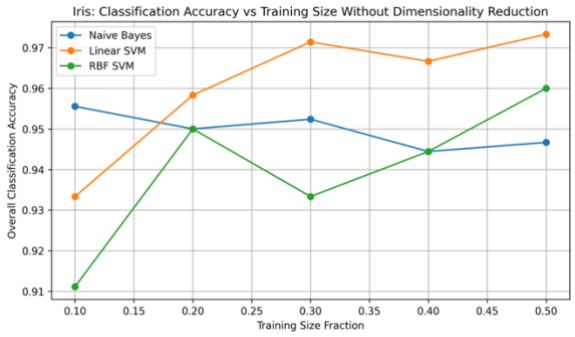

Figure 11. Classification accuracy versus training size for the Iris dataset without dimensionality reduction.

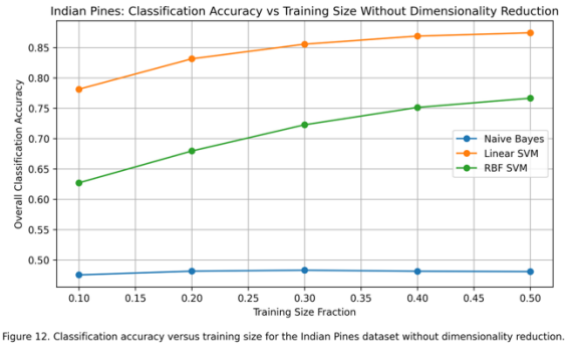

Figure 12. Classification accuracy versus training size for the Indian Pines dataset without dimensionality reduction.

Class	Naive Bayes	Linear SVM	RBF SVM
1	0.0938	0.0000	0.0000
2	0.3350	0.3110	0.3190
3	0.0000	0.0017	0.0000
4	0.0181	0.0301	0.0904
5	0.0296	0.2337	0.2278
6	0.8297	0.8023	0.9022
7	0.0000	0.0000	0.0000
8	0.9522	0.9821	0.9881
9	0.0000	0.0000	0.0000
10	0.0853	0.0000	0.0000
11	0.8232	0.9610	0.9459
12	0.1880	0.0000	0.1614
13	0.9091	0.8951	0.9021
14	0.9752	0.9786	0.9819
15	0.0630	0.0593	0.0259
16	0.0000	0.0000	0.0000

Table 1. Class-wise classification accuracy for Indian Pines dataset using PCA with 30% training size.

Analysis of Visualizations

Iris Dataset

Classification accuracy on the Iris dataset remains high across all methods, reflecting its strong inherent separability. Per Figures 7, 9, and 11, accuracy consistently exceeds 90%, regardless of training size or dimensionality reduction. LDA, when combined with classification, yields the highest accuracy by enhancing class separation before classification. PCA performs slightly below LDA because it does not use class labels. Even without dimensionality reduction, classification results remain strong, confirming the dataset's well-structured nature. Linear SVM typically achieves the best performance by effectively modeling the nearly linear decision boundaries in the dataset, whereas more complex kernels provide little additional benefit given the dataset’s simplicity.

Indian Pines Dataset

Classification results on the Indian Pines dataset underscore the importance of both dimensionality reduction and classifier selection. Figures 8, 10, and 12 show that accuracy does not always improve with dimensionality reduction; the highest accuracy actually occurs without dimensionality reduction using a Linear SVM. PCA provides moderate gains by reducing redundancy, and LDA can further enhance class separability, but neither method consistently outperforms the raw feature space. Simpler classifiers like Naive Bayes perform poorly without dimensionality reduction, while Linear SVM delivers the highest accuracy. RBF SVM performs well under dimensionality reduction by capturing nonlinear relationships but does not exceed Linear SVM in the raw feature space. The class-wise accuracy table highlights ongoing challenges: some classes remain difficult to classify, with near-zero accuracy, due to class imbalance and spectral similarity.

Appendix

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.io

from pathlib import Path
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC


def PullIrisReadingsFromSklearn():
    irisPacket = load_iris()
    irisMeasurementGrid = irisPacket.data
    irisClassLabels = irisPacket.target
    irisClassNames = irisPacket.target_names
    return irisMeasurementGrid, irisClassLabels, irisClassNames


def PullIndianPinesReadingsFromMatFiles():
    indianPinesSignalPacket = scipy.io.loadmat("indianR.mat")
    indianPinesTruthPacket = scipy.io.loadmat("indian_gth.mat")

    if "X" not in indianPinesSignalPacket:
        raise KeyError("Expected key 'X' in indianR.mat")

    if "gth" not in indianPinesTruthPacket:
        raise KeyError("Expected key 'gth' in indian_gth.mat")

    bandByPixelSignalGrid = indianPinesSignalPacket["X"]
    flattenedGroundTruthLabels = indianPinesTruthPacket["gth"].reshape(-1)

    pixelByBandSignalGrid = bandByPixelSignalGrid.T
    realClassMask = flattenedGroundTruthLabels > 0

    realPixelMeasurements = pixelByBandSignalGrid[realClassMask]
    realPixelLabels = flattenedGroundTruthLabels[realClassMask]

    return realPixelMeasurements, realPixelLabels


def NormalizeMeasurementColumns(measurementGrid, fittedScaler=None):
    if fittedScaler is None:
        scalerFittedHere = StandardScaler()
        scaledMeasurements = scalerFittedHere.fit_transform(measurementGrid)
        return scaledMeasurements, scalerFittedHere

    scaledMeasurements = fittedScaler.transform(measurementGrid)
    return scaledMeasurements, fittedScaler


def ComputePcaVarianceTrail(measurementGrid):
    scaledMeasurements, _ = NormalizeMeasurementColumns(measurementGrid)

    pcaReader = PCA()
    pcaReader.fit(scaledMeasurements)

    perComponentVariance = pcaReader.explained_variance_ratio_
    cumulativeVariance = np.cumsum(perComponentVariance)

    return perComponentVariance, cumulativeVariance


def BuildResultsFolder():
    resultsFolder = Path("results")
    resultsFolder.mkdir(exist_ok=True)
    return resultsFolder


def SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureCaption):
    safeFileStem = f"Figure_{figureNumber:02d}"
    figurePaper.savefig(resultsFolder / f"{safeFileStem}.png", dpi=300, bbox_inches="tight")


def PlotPcaVarianceTrail(measurementGrid, datasetName, figureNumber, figureCaption, resultsFolder):
    perComponentVariance, cumulativeVariance = ComputePcaVarianceTrail(measurementGrid)
    componentNumbers = np.arange(1, len(perComponentVariance) + 1)

    figurePaper, drawingArea = plt.subplots(figsize=(8, 5))
    drawingArea.plot(componentNumbers, perComponentVariance, marker="o", label="Individual explained variance")
    drawingArea.plot(componentNumbers, cumulativeVariance, marker="s", label="Cumulative explained variance")
    drawingArea.set_xlabel("Principal Component Number")
    drawingArea.set_ylabel("Explained Variance Ratio")
    drawingArea.set_title(f"{datasetName}: PCA Explained Variance")
    drawingArea.legend()
    drawingArea.grid(True)

    figurePaper.text(0.5, 0.01, figureCaption, ha="center", va="bottom", fontsize=10)
    figurePaper.tight_layout(rect=[0, 0.05, 1, 1])

    SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureCaption)
    plt.show()
```

```
def ProjectIntoTwoPcaAxes(measurementGrid):
    scaledMeasurements, _ = NormalizeMeasurementColumns(measurementGrid)
    twoAxisPcaProjector = PCA(n_components=2)
    twoAxisView = twoAxisPcaProjector.fit_transform(scaledMeasurements)
    return twoAxisView


def ProjectIntoTwoLdaAxes(measurementGrid, classLabels):
    scaledMeasurements, _ = NormalizeMeasurementColumns(measurementGrid)
    twoAxisLdaProjector = LinearDiscriminantAnalysis(n_components=2, solver="eigen")
    twoAxisView = twoAxisLdaProjector.fit_transform(scaledMeasurements, classLabels)
    return twoAxisView


def PlotClassColorMap(twoAxisView, classLabels, xAxisName, yAxisName, chartTitle, figureNumber, figureCaption, resultsFolder, classNameMap=None):
    figurePaper, drawingArea = plt.subplots(figsize=(7, 5))

    uniqueClassNumbers = np.unique(classLabels)
    colorPicture = drawingArea.scatter(
        twoAxisView[:, 0],
        twoAxisView[:, 1],
        c=classLabels,
        s=18,
        alpha=0.8,
        cmap="tab20"
    )

    drawingArea.set_xlabel(xAxisName)
    drawingArea.set_ylabel(yAxisName)
    drawingArea.set_title(chartTitle)
    drawingArea.grid(True)

    if len(uniqueClassNumbers) <= 10 and classNameMap is not None:
        legendHandles = []
        for classNumber in uniqueClassNumbers:
            normalizedColorSpot = colorPicture.norm(classNumber)
            classColor = colorPicture.cmap(normalizedColorSpot)
            legendLabel = classNameMap[int(classNumber)]
            legendHandles.append(
                plt.Line2D(
                    [0],
                    [0],
                    marker="o",
                    color="w",
                    label=legendLabel,
                    markerfacecolor=classColor,
                    markersize=7
                )

            )

        drawingArea.legend(handles=legendHandles, title="Class", loc="best")
    else:
        colorBar = figurePaper.colorbar(colorPicture, ax=drawingArea)
        colorBar.set_label("Class Label")
        colorBar.set_ticks(uniqueClassNumbers)

    figurePaper.text(0.5, 0.01, figureCaption, ha="center", va="bottom", fontsize=10)
    figurePaper.tight_layout(rect=[0, 0.05, 1, 1])

    SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureCaption)
    plt.show()


def PrepareTrainTestViews(measurementGrid, classLabels, trainingShare, shrinkMethodName=None, keptDirectionCount=2):
    trainingMeasurements, testingMeasurements, trainingClassLabels, testingClassLabels = train_test_split(
        measurementGrid,
        classLabels,
        train_size=trainingShare,
        stratify=classLabels,
        random_state=42
    )

    scaledTrainingMeasurements, fittedScaler = NormalizeMeasurementColumns(trainingMeasurements)
    scaledTestingMeasurements, _ = NormalizeMeasurementColumns(testingMeasurements, fittedScaler)

    if shrinkMethodName is None:
        return scaledTrainingMeasurements, scaledTestingMeasurements, trainingClassLabels, testingClassLabels

    if shrinkMethodName == "PCA":
        pcaShrinker = PCA(n_components=keptDirectionCount)
        shrunkenTrainingMeasurements = pcaShrinker.fit_transform(scaledTrainingMeasurements)
        shrunkenTestingMeasurements = pcaShrinker.transform(scaledTestingMeasurements)
        return shrunkenTrainingMeasurements, shrunkenTestingMeasurements, trainingClassLabels, testingClassLabels

    if shrinkMethodName == "LDA":
        ldaShrinker = LinearDiscriminantAnalysis(n_components=keptDirectionCount, solver="eigen")
        shrunkenTrainingMeasurements = ldaShrinker.fit_transform(scaledTrainingMeasurements, trainingClassLabels)
        shrunkenTestingMeasurements = ldaShrinker.transform(scaledTestingMeasurements)
        return shrunkenTrainingMeasurements, shrunkenTestingMeasurements, trainingClassLabels, testingClassLabels

    raise ValueError("shrinkMethodName must be None, 'PCA', or 'LDA'.")


def GatherClassifiers():
    return {
        "Naive Bayes": GaussianNB(),
        "Linear SVM": SVC(kernel="linear"),
        "RBF SVM": SVC(kernel="rbf")
    }


def MeasureAccuracyAcrossTrainingShares(measurementGrid, classLabels, shrinkMethodName=None, keptDirectionCount=2):
    trainingShareSteps = [0.10, 0.20, 0.30, 0.40, 0.50]

    classifierAccuracyTimeline = {
        "Naive Bayes": [],
        "Linear SVM": [],
```

```
        "RBF SVM": []
    }

    for trainingShare in trainingShareSteps:
        readyTrainingMeasurements, readyTestingMeasurements, trainingClassLabels, testingClassLabels = PrepareTrainTestViews(
            measurementGrid=measurementGrid,
            classLabels=classLabels,
            trainingShare=trainingShare,
            shrinkMethodName=shrinkMethodName,
            keptDirectionCount=keptDirectionCount
        )

        for classifierName, classifierMachine in GatherClassifiers().items():
            classifierMachine.fit(readyTrainingMeasurements, trainingClassLabels)
            predictedLabels = classifierMachine.predict(readyTestingMeasurements)
            overallAccuracy = accuracy_score(testingClassLabels, predictedLabels)
            classifierAccuracyTimeline[classifierName].append(overallAccuracy)

    return trainingShareSteps, classifierAccuracyTimeline


def PlotAccuracyGrowthStory(trainingShares, classifierAccuracyTimeline, chartTitle, figureNumber, figureCaption, resultsFolder):
    figurePaper, drawingArea = plt.subplots(figsize=(8, 5))

    for classifierName, accuracyList in classifierAccuracyTimeline.items():
        drawingArea.plot(trainingShares, accuracyList, marker="o", label=classifierName)

    drawingArea.set_xlabel("Training Size Fraction")
    drawingArea.set_ylabel("Overall Classification Accuracy")
    drawingArea.set_title(chartTitle)
    drawingArea.legend()
    drawingArea.grid(True)

    figurePaper.text(0.5, 0.01, figureCaption, ha="center", va="bottom", fontsize=10)
    figurePaper.tight_layout(rect=[0, 0.05, 1, 1])

    SaveFinishedFigure(figurePaper, resultsFolder, figureNumber, figureCaption)
    plt.show()


def BuildIndianPinesThirtyPercentPcaClassBreakdown(measurementGrid, classLabels, keptDirectionCount=2):
    readyTrainingMeasurements, readyTestingMeasurements, trainingClassLabels, testingClassLabels = PrepareTrainTestViews(
        measurementGrid=measurementGrid,
        classLabels=classLabels,
        trainingShare=0.30,
        shrinkMethodName="PCA",
        keptDirectionCount=keptDirectionCount
    )

    classNumbersInOrder = np.unique(classLabels)
    classAccuracyBook = {}

    for classifierName, classifierMachine in GatherClassifiers().items():
        classifierMachine.fit(readyTrainingMeasurements, trainingClassLabels)
        predictedLabels = classifierMachine.predict(readyTestingMeasurements)

        confusionMatrixBox = confusion_matrix(
            testingClassLabels,
            predictedLabels,
            labels=classNumbersInOrder
        )

        classTotals = confusionMatrixBox.sum(axis=1)
        classAccuracyList = np.divide(
            confusionMatrixBox.diagonal(),
            classTotals,
            out=np.zeros_like(classTotals, dtype=float),
            where=classTotals != 0
        )


        classAccuracyBook[classifierName] = classAccuracyList

    classAccuracyTable = pd.DataFrame(
        {

            "Class": classNumbersInOrder.astype(int),
            "Naive Bayes": classAccuracyBook["Naive Bayes"],
            "Linear SVM": classAccuracyBook["Linear SVM"],
            "RBF SVM": classAccuracyBook["RBF SVM"]
        }

    )

    return classAccuracyTable


def SaveIndianPinesClassTable(classAccuracyTable, resultsFolder):
    csvPath = resultsFolder / "Table_1_Indian_Pines_PCA_30_Percent_Class-wise_Accuracy.csv"
    classAccuracyTable.to_csv(csvPath, index=False)

    figurePaper, drawingArea = plt.subplots(figsize=(8.5, max(4, 0.4 * len(classAccuracyTable) + 1.6)))
    drawingArea.axis("off")

    formattedTable = classAccuracyTable.copy()
    for columnName in ["Naive Bayes", "Linear SVM", "RBF SVM"]:
        formattedTable[columnName] = formattedTable[columnName].map(lambda value: f"{value:.4f}")

    tablePicture = drawingArea.table(
        cellText=formattedTable.values,
        colLabels=formattedTable.columns,
        loc="center",
        cellLoc="center"
    )

    tablePicture.auto_set_font_size(False)
    tablePicture.set_fontsize(9)
```

```
    tablePicture.scale(1, 1.25)

    tableCaption = "Table 1. Class-wise classification accuracy for Indian Pines dataset using PCA with 30% training size."
    figurePaper.text(0.5, 0.01, tableCaption, ha="center", va="bottom", fontsize=10)
    figurePaper.tight_layout(rect=[0, 0.05, 1, 1])

    figurePaper.savefig(resultsFolder / "Table_1_Indian_Pines_PCA_30_Percent_Class-wise_Accuracy.png", dpi=300, bbox_inches="tight")


def RunComparativeDimensionalityStudy():
    resultsFolder = BuildResultsFolder()
    keptDirectionCount = 2

    figureCaptions = {
        1: "Figure 1. PCA explained variance for the Iris dataset.",
        2: "Figure 2. PCA explained variance for the Indian Pines dataset.",
        3: "Figure 3. PCA two-dimensional projection for the Iris dataset.",
        4: "Figure 4. PCA two-dimensional projection for the Indian Pines dataset.",
        5: "Figure 5. LDA two-dimensional projection for the Iris dataset.",
        6: "Figure 6. LDA two-dimensional projection for the Indian Pines dataset.",
        7: "Figure 7. Classification accuracy versus training size for the Iris dataset using PCA plus classification.",
        8: "Figure 8. Classification accuracy versus training size for the Indian Pines dataset using PCA plus classification.",
        9: "Figure 9. Classification accuracy versus training size for the Iris dataset using LDA plus classification.",
        10: "Figure 10. Classification accuracy versus training size for the Indian Pines dataset using LDA plus classification.",
        11: "Figure 11. Classification accuracy versus training size for the Iris dataset without dimensionality reduction.",
        12: "Figure 12. Classification accuracy versus training size for the Indian Pines dataset without dimensionality reduction."
    }

    irisFeatureGrid, irisClassLabels, irisClassNames = PullIrisReadingsFromSklearn()
    indianPinesFeatureGrid, indianPinesClassLabels = PullIndianPinesReadingsFromMatFiles()

    irisClassNameMap = {classNumber: className for classNumber, className in enumerate(irisClassNames)}

    PlotPcaVarianceTrail(
        measurementGrid=irisFeatureGrid,
        datasetName="Iris",
        figureNumber=1,
        figureCaption=figureCaptions[1],
        resultsFolder=resultsFolder
    )

    PlotPcaVarianceTrail(
        measurementGrid=indianPinesFeatureGrid,
        datasetName="Indian Pines",
        figureNumber=2,
        figureCaption=figureCaptions[2],
        resultsFolder=resultsFolder
    )

    irisPcaProjection = ProjectIntoTwoPcaAxes(irisFeatureGrid)
    PlotClassColorMap(
        twoAxisView=irisPcaProjection,
        classLabels=irisClassLabels,
        xAxisName="PC1",
        yAxisName="PC2",
        chartTitle="Iris: PCA First Two Directions",
        figureNumber=3,
        figureCaption=figureCaptions[3],
        resultsFolder=resultsFolder,
        classNameMap=irisClassNameMap
    )

    indianPinesPcaProjection = ProjectIntoTwoPcaAxes(indianPinesFeatureGrid)
    PlotClassColorMap(
        twoAxisView=indianPinesPcaProjection,
        classLabels=indianPinesClassLabels,
        xAxisName="PC1",
        yAxisName="PC2",
        chartTitle="Indian Pines: PCA First Two Directions",
        figureNumber=4,
        figureCaption=figureCaptions[4],
        resultsFolder=resultsFolder
    )

    irisLdaProjection = ProjectIntoTwoLdaAxes(irisFeatureGrid, irisClassLabels)
    PlotClassColorMap(
        twoAxisView=irisLdaProjection,
        classLabels=irisClassLabels,
        xAxisName="LD1",
        yAxisName="LD2",
        chartTitle="Iris: LDA First Two Directions",
        figureNumber=5,
        figureCaption=figureCaptions[5],
        resultsFolder=resultsFolder,
        classNameMap=irisClassNameMap
    )

    indianPinesLdaProjection = ProjectIntoTwoLdaAxes(indianPinesFeatureGrid, indianPinesClassLabels)
    PlotClassColorMap(
        twoAxisView=indianPinesLdaProjection,
        classLabels=indianPinesClassLabels,
        xAxisName="LD1",
        yAxisName="LD2",
        chartTitle="Indian Pines: LDA First Two Directions",
        figureNumber=6,
        figureCaption=figureCaptions[6],
        resultsFolder=resultsFolder
    )

    irisPcaTrainingShares, irisPcaAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=irisFeatureGrid,
        classLabels=irisClassLabels,
        shrinkMethodName="PCA",
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=irisPcaTrainingShares,
        classifierAccuracyTimeline=irisPcaAccuracyTimeline,
        chartTitle="Iris: PCA Plus Classification Accuracy vs Training Size",
        figureNumber=7,
```

```
        figureCaption=figureCaptions[7],
        resultsFolder=resultsFolder
    )

    indianPinesPcaTrainingShares, indianPinesPcaAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=indianPinesFeatureGrid,
        classLabels=indianPinesClassLabels,
        shrinkMethodName="PCA",
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=indianPinesPcaTrainingShares,
        classifierAccuracyTimeline=indianPinesPcaAccuracyTimeline,
        chartTitle="Indian Pines: PCA Plus Classification Accuracy vs Training Size",
        figureNumber=8,
        figureCaption=figureCaptions[8],
        resultsFolder=resultsFolder
    )

    irisLdaTrainingShares, irisLdaAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=irisFeatureGrid,
        classLabels=irisClassLabels,
        shrinkMethodName="LDA",
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=irisLdaTrainingShares,
        classifierAccuracyTimeline=irisLdaAccuracyTimeline,
        chartTitle="Iris: LDA Plus Classification Accuracy vs Training Size",
        figureNumber=9,
        figureCaption=figureCaptions[9],
        resultsFolder=resultsFolder
    )

    indianPinesLdaTrainingShares, indianPinesLdaAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=indianPinesFeatureGrid,
        classLabels=indianPinesClassLabels,
        shrinkMethodName="LDA",
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=indianPinesLdaTrainingShares,
        classifierAccuracyTimeline=indianPinesLdaAccuracyTimeline,
        chartTitle="Indian Pines: LDA Plus Classification Accuracy vs Training Size",
        figureNumber=10,
        figureCaption=figureCaptions[10],
        resultsFolder=resultsFolder
    )

    irisRawTrainingShares, irisRawAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=irisFeatureGrid,
        classLabels=irisClassLabels,
        shrinkMethodName=None,
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=irisRawTrainingShares,
        classifierAccuracyTimeline=irisRawAccuracyTimeline,
        chartTitle="Iris: Classification Accuracy vs Training Size Without Dimensionality Reduction",
        figureNumber=11,
        figureCaption=figureCaptions[11],
        resultsFolder=resultsFolder
    )

    indianPinesRawTrainingShares, indianPinesRawAccuracyTimeline = MeasureAccuracyAcrossTrainingShares(
        measurementGrid=indianPinesFeatureGrid,
        classLabels=indianPinesClassLabels,
        shrinkMethodName=None,
        keptDirectionCount=keptDirectionCount
    )

    PlotAccuracyGrowthStory(
        trainingShares=indianPinesRawTrainingShares,
        classifierAccuracyTimeline=indianPinesRawAccuracyTimeline,
        chartTitle="Indian Pines: Classification Accuracy vs Training Size Without Dimensionality Reduction",
        figureNumber=12,
        figureCaption=figureCaptions[12],
        resultsFolder=resultsFolder
    )

    classAccuracyTable = BuildIndianPinesThirtyPercentPcaClassBreakdown(
        measurementGrid=indianPinesFeatureGrid,
        classLabels=indianPinesClassLabels,
        keptDirectionCount=keptDirectionCount
    )

    SaveIndianPinesClassTable(classAccuracyTable, resultsFolder)

    print(f"\nSaved outputs to: {resultsFolder.resolve()}")

if __name__ == "__main__":
    RunComparativeDimensionalityStudy()
```